An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary retains comparisons in directed acyclic graphs and represents possible orders by a region of real positions satisfying interval bounds and the recorded inequalities. It estimates the fraction of this region's volume retained by each answer through deterministic calculations on bounded graph neighborhoods. Moving items into ordered subproblems controls graph degrees and directed path lengths; a geometric analysis shows that these moves preserve the optimal asymptotic bound.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

Deterministic adversaries from graph entropy. Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These

accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir, and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the earlier adversaries. Section 3 gives our data structure and comparison procedure. Items are stored at nodes of a rooted tree, with a directed acyclic graph recording comparisons within each node. Interval bounds and the graph's inequalities define a region of real positions representing the possible orders there. The adversary estimates the fraction of this region's volume retained by each answer. Three technical ingredients make this possible.

Deterministic estimates from bounded neighborhoods. After propagating interval bounds through the full graph, we average finite computations on a bounded neighborhood. Each step changes one position using a choice from a fixed finite set. Randomizing these choices would preserve the continuous uniform distribution exactly. Comparing this process with a computation started from the effective lower endpoints shows that the finite average estimates the desired probability. The degree and path-length bounds control how far differences can propagate (Section 6).

Choosing positions with large remaining volume. Moving an item into a child fixes its position in the parent. We choose a binary fraction with the fewest digits in a suitable interval, obtained from approximate quartiles or from the item's interval bounds. Geometric inequalities show that this choice leaves enough volume for the other items (Section 5.3).

Bounding the cost of graph simplification. Section 4 compares the volume gained by moving items with the probability of choosing their children in a counting experiment. Items confined to very short intervals and groups of items from long directed paths supply enough gain to pay for these choices. Moves triggered by high degree are charged to the comparisons whose arcs they remove. The experiment then relates the final state to the single total order determined by the sorter.

Finally, Section 7 lets the degree and path-length bounds grow slowly with n, keeping the computation polynomial as the comparison coefficient approaches one.

Appendix A proves the density estimates, and Appendix B collects the parameters. The manuscript source is available online.[1]

2 Related Work

2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

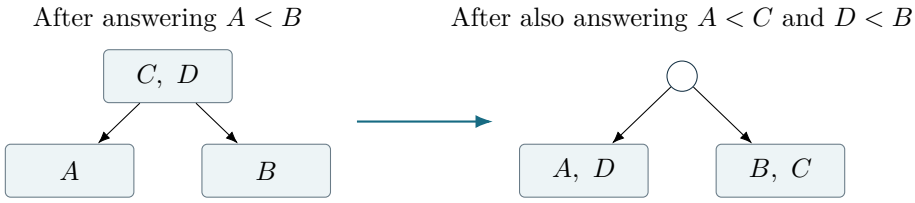


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this

interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

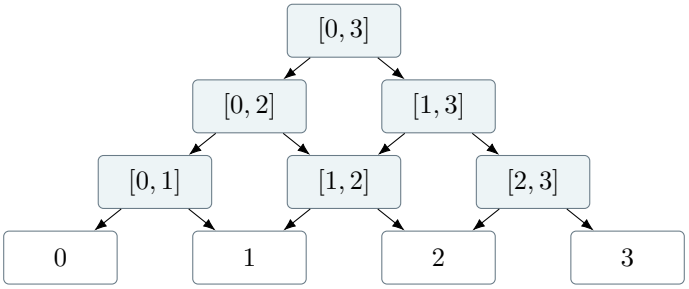


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

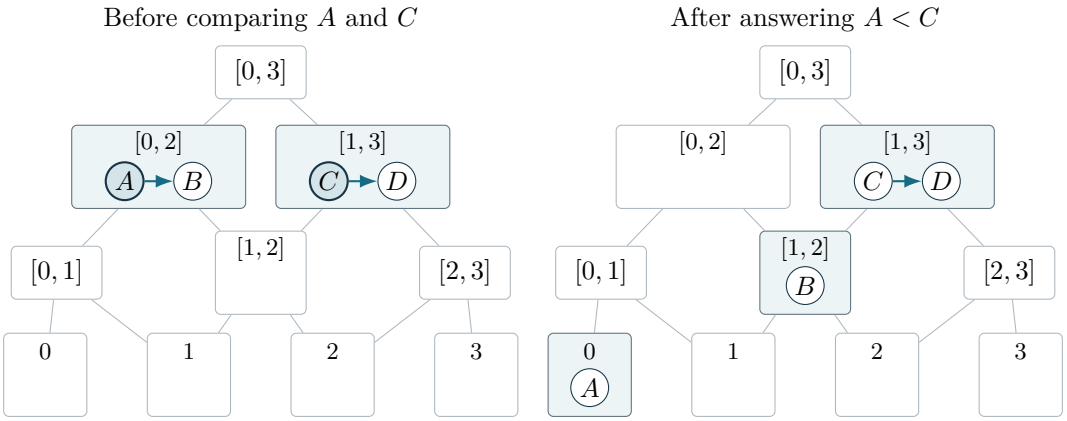


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

3 The adversary

We describe the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [6] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of b and c undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in that node's DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. We call this its *in-node DAG*. Figure 4 shows an example.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields a_x, b_x restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. The bounds a_x, b_x are fractions whose denominators are powers of two; we call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In an in-node DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. Each answered comparison between free items contributes at most one arc.

A *feasible assignment* at a node is a choice of real values for its position variables satisfying $a_x < X_x < b_x$ for every item x and $X_x < X_y$ for every arc $x \to y$ in its in-node DAG.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables X_x are unassigned, and the root's in-node DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

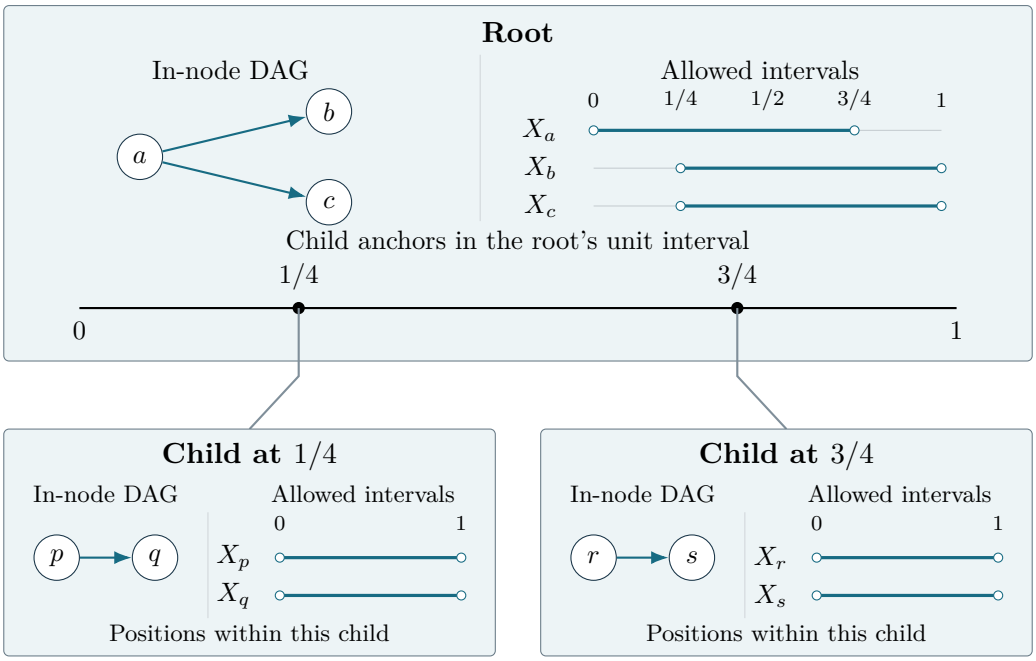


Figure 4: A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 4, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. We call the sequence of these answers the *transcript*.

3.2 Answering a comparison

Algorithm 1 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare A and B, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors determine the answer immediately. In the other cases, compute the bounds implied by the in-node DAG. If there is an arc $y \to x$, every lower bound on X_y is also a lower bound on X_x; upper bounds propagate in the reverse direction. Write $y \preceq x$ when $y = x$ or a directed path leads from y to x. The resulting *effective interval* is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{1}$$

These endpoints can be computed in two passes through the in-node DAG: lower endpoints in topological order, then upper endpoints in reverse order. We compute them when needed, keeping the record's bounds a_x, b_x as introduced by answers and refinements.

For a free coordinate X_x and an anchor c, the answer is determined if $c \le \ell_x$ or $c \ge r_x$. For two free coordinates, a directed path between them determines their order; so do effective intervals

that are disjoint or meet only at an endpoint. Return this answer immediately. Lemma 5.2 proves that these tests detect every comparison fixed by the represented orders, including every previously answered pair.

For an undecided comparison, let p be the probability that A precedes B when positions at this node are assigned uniformly subject to its interval bounds and in-node DAG inequalities. The subroutine `Estimate` returns a deterministic estimate $\widehat{p}$. Fix a rational error tolerance $0 < \eta < 1/4$. Choose $A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise. For two free items, the event being estimated is $X_A < X_B$. For a free item x and an anchor c, estimate $X_x < c$, taking the complement if the queried order is reversed. An answer between two free items adds an arc to the in-node DAG; an answer involving an anchor adds an interval bound.

Before using the estimator, we ensure that every queried free coordinate has effective width at least $1/R$, where $R \geq 1$ is a fixed rational threshold. Call such a coordinate *wide*, and a narrower one *narrow*. A narrow queried item is moved into a child, and the query is routed again. This move is called a *refinement*, denoted `Refine`(x); Algorithm 2 in Section 3.3 gives the operation.

We also control the in-node DAGs using integers $k, H \geq 2$. Throughout the interaction, each vertex has at most k neighbors, counting both incoming and outgoing arcs, and each directed path has fewer than $2H$ vertices. Before estimating an undecided comparison, we first remove any path of H vertices by refining its entire list of items. If a queried free item has degree k, we refine that item too. Each of these steps restarts the query. When the estimator is reached, the queried vertices have degree below k and the current node has no H-vertex path. Adding one arc therefore preserves the bounds needed throughout the interaction: degrees rise by at most one, and a new path joins two old paths.

The following lemma supplies the accuracy needed by the comparison rule and anchor selection. Section 6 gives the estimator and proves the lemma.

Lemma 3.1 (Probability estimation)**.** *Suppose a node admits a feasible assignment, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. Under the uniform distribution on these assignments,* `Estimate` *deterministically computes, with error at most η:*

(i) $\mathbb{P}(X_x < c)$*, when x is wide and c is dyadic;*
(ii) $\mathbb{P}(X_x < X_y)$*, when at least one of x, y is wide.*

For a wide coordinate x, a companion routine `Marginal` *returns a finite list of positions. For every real threshold c, the fraction of this list below c differs from $\mathbb{P}(X_x < c)$ by at most η.*

Algorithm 1: `Compare`(A, B): answer a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
while *true* **do**
  $v \leftarrow$ the root
  while *A and B lie in the same child subtree of v* **do**
    $v \leftarrow$ that child
  if *A and B lie in different child subtrees of v* **then**
    return the left-to-right order of their child anchors
  Compute effective intervals at v using (1)
  if *effective intervals or a directed path in the in-node DAG at v determine the answer* **then**
    return the determined answer
  if *the in-node DAG at v has a directed path on H vertices* **then**
    $L \leftarrow$ the list of labels on the selected path
    foreach $x \in L$ *in label order* **do**
      `Refine`(x)
    continue
  if *a queried free item has degree k in the in-node DAG at v* **then**
    `Refine`(the selected such item)
    continue
  if *a queried free coordinate is narrow* **then**
    `Refine`(the selected such item)
    continue
  $\widehat{p} \leftarrow$ `Estimate`$(v, A < B, \eta)$
  answer $\leftarrow A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise
  Store *answer* as an arc in the in-node DAG at v or as an interval bound
  return *answer*

Every restart refines at least one item. Its new coordinate in the child is isolated and has interval $(0, 1)$. Further refinements cannot give that coordinate an incident arc or narrow its interval, so it cannot need refinement again during this call. Thus a call makes at most n refinements; Section 5.4 gives the full proof.

For deterministic selection, choose the smallest eligible item label and the lexicographically first eligible directed path. Complete each selected path's list even if earlier refinements break the path.

3.3 Moving an item into a child

Algorithm 2 gives `Refine`(x), which moves x from its current node into a child. It computes x's effective interval and calls `ChooseAnchor` to select a position c inside it.

We select anchor positions using a fixed infinite binary tree, the *implicit anchor tree*, shown in Figure 5. The same tree is used at every node of the adversary's tree. Each anchor-tree node u has an interval $I(u) = [a, b]$ and a position $c(u) = (a + b)/2$; write $|I(u)| = b - a$. The root has interval $[0, 1]$; the left and right children of u have intervals $[a, c(u)]$ and $[c(u), b]$, respectively.

At depth m, the 2^m nodes have intervals of length 2^{-m} and positions

$$c = (i + \tfrac{1}{2})2^{-m}, \qquad i = 0, \ldots, 2^m - 1. \tag{2}$$

We call m the *anchor depth*, always measured in the anchor tree. These positions are dyadic

rationals, so refinement keeps the interval endpoints a_x, b_x dyadic. A finite path from the anchor tree's root specifies an anchor. Algorithm 3 in Section 5.3 gives the rule for choosing a central interval and selecting its shallowest anchor. Equivalently, this is the binary fraction in that interval with the fewest fractional digits.

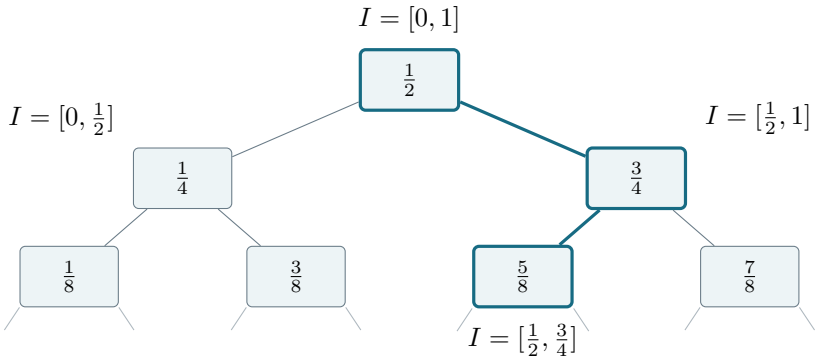


Figure 5: The implicit anchor tree. The highlighted path selects 5/8 at depth 2.

Fixing $X_x = c$ places x in the child block anchored at c. The arcs incident to x in that in-node DAG become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, and retain the new bounds on its neighbors. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 6 illustrates reuse of an existing child.

Algorithm 2: `Refine`(x): move an item into a child.

Input: A free item x and the current adversary state.
Output: The updated state, with x stored in a child of its former node.
$v \leftarrow$ the node storing x
Compute x's effective interval using (1)
$c \leftarrow$ `ChooseAnchor`(x)
foreach *arc* $y \to x$ *in the in-node DAG at* v **do**
  $b_y \leftarrow \min(b_y, c)$
foreach *arc* $x \to z$ *in the in-node DAG at* v **do**
  $a_z \leftarrow \max(a_z, c)$
Remove x's record from v and its vertex and incident arcs from the in-node DAG at v
if v *has no child anchored at* c **then**
  Create a child of v at c, with no records and an empty in-node DAG
$u \leftarrow$ the child of v anchored at c
Insert a record $(x, X_x, 0, 1)$ at u, with a fresh, unassigned position variable X_x
Insert x as an isolated vertex in the in-node DAG at u

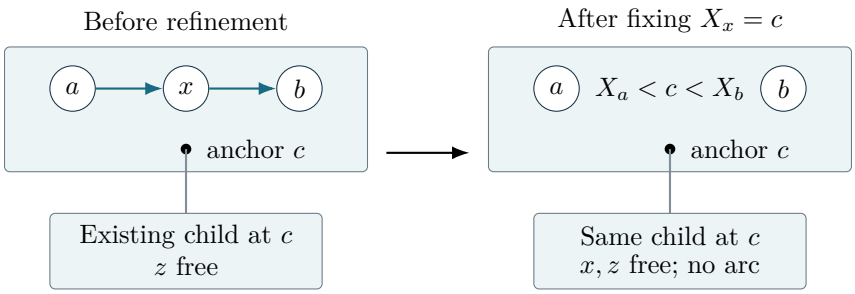


Figure 6: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.3 proves that this operation preserves all answered comparisons.

Refinement can impose additional order relations. For example, suppose the root contains A, B, C with only $A < B$ recorded. Refining A at $1/2$ restricts B to $(1/2, 1)$. Refining C into the same child then leaves both $A < C < B$ and $C < A < B$ possible, while fixing the additional relation $C < B$. Section 4 shows that we can afford this loss of possible orders while retaining the optimal asymptotic comparison bound.

Table 1 collects the thresholds used by Algorithms 1 and 2 above. In the next section, we explain why the adversary forces any correct sorter to perform many comparisons.

4 Why the adversary forces many comparisons

Recall from Algorithm 1 that a larger degree threshold k lets the adversary retain more comparisons involving each item before moving it into a child. Increasing k lets the adversary force a comparison count closer to $n \log n$. The following bound quantifies this dependence.

Theorem 4.1 (Comparison bound)**.** *For every power of two $k \geq 64$, the adversary of Algorithm 1, with the remaining parameter choices in Table 1, is deterministic and legal, and every call terminates. Every correct sorter makes at least*

$$\frac{n \log n - 3n}{1 + (2 \log k + 17)/k} \tag{3}$$

comparisons.

For this section, fix a power of two $k \geq 64$. Section 7 chooses k as a function of n so that it grows without bound while the adversary's running time remains polynomial.

The proof uses a random experiment that chooses a node and a position for each item. At termination, all assignments admitted by the state have the same relative order, so their probability is at most $1/n!$. We bound this probability from below using the feasible volumes within nodes and the probabilities of the child choices made during refinement. Relating those child choices to the volume gained during refinement will leave only a small additional loss per comparison.

4.1 Volume changes and anchor selection

We next introduce the definitions required to state Lemma 4.2 and prove Theorem 4.1.

To assess an anchor choice c for x, we compare the volume left for the other items after fixing $X_x = c$ with the node's original feasible volume. Their ratio is the density $f_x(c)$. Let v be the node containing x; its feasible assignments form the region

$$K_v = \{X : a_x < X_x < b_x \text{ for every free } x, \quad X_x < X_y \text{ for every in-node arc } x \to y\}. \tag{4}$$

Write $V_v = \mathrm{vol}(K_v)$, and let $S_x(c)$ be the volume of the section obtained by fixing $X_x = c$, measured in the remaining coordinates. Under the uniform distribution on K_v, the density of X_x is

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\,dt.$$

To track volume across the adversary's tree, take the product of the node regions. Each item has a position variable at exactly one node, so this product has dimension n and volume

$$Z = \prod_v V_v. \tag{5}$$

A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \le 1$.

Choose the estimation error to be

$$\eta = \frac{1}{100k}. \tag{6}$$

By Lemma 3.1, each estimated answer retains at least $1/2 - \eta$ of its node's volume. Its decrease in $\log Z$ is therefore at most

$$a_\eta = -\log(1/2 - \eta) \le 1 + \frac{1}{2k}. \tag{7}$$

For the inequality, use $2^{-1/(2k)} \le 1 - 1/(4k) \le 1 - 1/(50k) = 1 - 2\eta$. An answer already determined by the stored constraints leaves Z unchanged.

For refinement we need a lower bound on $f_x(c)$ in terms of the anchor's depth in the implicit anchor tree. Put

$$B = (k+1)^{2H+1}. \tag{8}$$

In an in-node DAG of degree at most k and directed paths of at most $2H$ vertices, every coordinate comparable with x lies within undirected distance $2H$ of x. Thus B bounds the number of other coordinates comparable with x.

Lemma 4.2 (Anchor selection)**.** *Suppose a node's feasible region has positive volume, its interval endpoints are dyadic, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. For $R = 2^{2kB}$ and rational $0 < \eta < 1/32$,* `ChooseAnchor` *deterministically returns an anchor $c \in (\ell_x, r_x)$ at depth m in the implicit anchor tree satisfying*

$$\log f_x(c) \ge (1 - 1/k)m - 4. \tag{9}$$

If x is narrow, then

$$\log f_x(c) > k. \tag{10}$$

Section 5.3.3 proves Lemma 4.2 and explains the choice of R. The density bound controls the change in Z through the following identity. Refinement fixes X_x in the parent and gives x a fresh unit coordinate in the child.

Lemma 4.3 (Refinement identity)**.** *Suppose every node region has positive volume. Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{11}$$

Section 5.2 proves this identity, including when the child already contains items. Thus the change in $\log Z$ caused by this refinement is

$$g = \log f_x(c). \tag{12}$$

We call g the refinement's *volume gain*; it can be negative.

To apply the probability and anchor estimates throughout a call, we need positive node volumes and bounds on the in-node DAGs. We also need each call to return an answer consistent with the earlier answers.

Proposition 4.4 (Consistency and termination)**.** *With the parameter choices of Theorem 4.1, every reached state has positive node volumes and represents only orders consistent with the transcript. Every comparison call terminates and performs at most n refinements. Every in-node DAG has degree at most k and directed paths of fewer than $2H$ vertices throughout the interaction.*

Section 5.4 proves Proposition 4.4 from the comparison and refinement rules, using Lemmas 3.1 and 4.2.

4.2 The cost of choosing a child

The product Z measures freedom within individual nodes. For example, two items stored alone in separate children can each retain the full interval $(0, 1)$, giving $Z = 1$, even though the child anchors fix their order. To account for the order determined by the tree, we use a random experiment on a fixed state.

Start at the root of the adversary's tree. At a visited node, stop with probability $1/2$. Otherwise, choose an anchor by a walk in the implicit anchor tree: at each node of that tree, select its position with probability $1/k$; otherwise, move to the left or right child with equal probability. If the selected anchor has an existing child in the adversary's tree, continue the experiment in that child. Otherwise, stop at the current node. Whenever the experiment stops in the adversary's tree, choose a uniform coordinate in $(0, 1)$ there.

The walk selecting an anchor terminates almost surely. A particular anchor at depth m requires m specified left or right choices, so an existing child at that anchor is entered with probability

$$w(m) = \frac{1}{2k}\left(\frac{1 - 1/k}{2}\right)^m = \frac{2^{-\gamma m}}{2k}, \qquad \gamma = 1 - \log(1 - 1/k). \tag{13}$$

The probability of stopping at each visited node of the adversary's tree is at least $1/2$. Since the state has a finite tree, the experiment produces a node and a coordinate almost surely.

Sample one such position independently for each of the n labels. The tree orders these positions by the same rule as in Section 3: compare a coordinate with the child anchors, and order positions inside one child recursively. Ties have probability zero, and all labels use the same distribution. Permuting the labels therefore shows that each relative order has probability $1/n!$.

For the counting argument, measure a child choice by its contribution to the negative logarithm of the probability:

$$\ell(m) = -\log w(m) = \gamma m + \log k + 1. \tag{14}$$

The next lemma expresses the final restriction to one order in terms of these costs.

Lemma 4.5 (Bound at termination)**.** *Suppose the transcript determines a unique total order. If m_r is the anchor depth used by refinement r, then*

$$\log(n!) \le n - \log Z + \sum_r \ell(m_r). \tag{15}$$

Proof. Consider the event that every label stops at the node where its record is stored, and that the sampled coordinates satisfy all the in-node constraints. Conditional on the chosen stopping nodes, the coordinates are independent and uniform, so the probability of satisfying the constraints is Z.

The stopping probabilities contribute at least 2^{-n}. Since records only move downward, their required paths, counted separately for each label, contain exactly the edge traversals made by refinements. The child choices therefore contribute $\prod_r w(m_r)$. The event has probability at least

$$Z\,2^{-n}\prod_r w(m_r) = Z\,2^{-n-\sum_r \ell(m_r)}.$$

By Proposition 4.4, the event produces an order consistent with the transcript. The unique such order has probability $1/n!$ in the random experiment. Comparing these probabilities and taking logarithms proves (15). □

4.3 Comparing refinement costs and volume gains

We now bound each cost $\ell(m)$ using the volume gain g of the same refinement. For a narrow coordinate, Lemma 4.2 gives $g > k$, even when the chosen anchor has small depth. Path batches obtain a large total gain because their anchors must occupy distinct nodes of the implicit anchor tree.

Choose the path threshold

$$H = 2^k. \tag{16}$$

The following bound supplies the depth sum needed for a complete batch.

Lemma 4.6 (Sum of anchor depths along a path)**.** *The H labels of a selected path in an in-node DAG enter distinct children of their parent in the adversary's tree. If their anchors have depths $m_1, \dots, m_H$ in the implicit anchor tree, then*

$$\sum_{i=1}^{H} m_i \geq H(k-2). \tag{17}$$

Section 5.5 proves the distinct-child claim and counts the available anchors. This is why Algorithm 1 finishes the whole selected path even when earlier refinements have broken it.

Outside a path batch, the algorithm refines a queried item because its vertex has degree k or its coordinate is narrow. When both hold, count it as a degree refinement. To compare costs with gains, set

$$\lambda = 1 + \frac{\log k + 10}{k}. \tag{18}$$

The factor λ approaches one as k increases.

Lemma 4.7 (Cost of refinement)**.** *A degree refinement with anchor depth m and volume gain g satisfies*

$$\ell(m) \leq \lambda g + \log k + 6.$$

A narrow refinement satisfies $\ell(m) \leq \lambda g$. For a complete path batch with depths m_i and gains g_i,

$$\sum_{i=1}^{H} \ell(m_i) \leq \lambda \sum_{i=1}^{H} g_i.$$

Proof. Write $j = \log k$. We use

$$\gamma - 1 = -\log(1 - 1/k) \leq \frac{1}{(k-1)\ln 2} \leq \frac{3}{2k} \qquad (k \geq 64). \tag{19}$$

The choice of λ gives $\lambda \le 5/4$ and

$$\delta := \lambda(1-1/k) - \gamma \ge \frac{j+15/2}{k} - \frac{j+10}{k^2} > 0.$$

By the anchor guarantee (9), every refinement satisfies

$$\lambda g - \ell(m) \ge \delta m - 4\lambda - j - 1. \tag{20}$$

Since $m \ge 0$ and $4\lambda \le 5$, this proves the degree-refinement bound.

For a narrow refinement, use $g > k$ and $m \le (g+4)/(1-1/k)$ to obtain

$$\begin{aligned}(1-1/k)\big(\lambda g - \ell(m)\big) &\ge \delta g - 4\gamma - (1-1/k)(j+1)\\ &> \delta k - 4\gamma - (1-1/k)(j+1)\\ &\ge \frac{5}{2} - \frac{15}{k} > 0.\end{aligned}$$

Thus its volume gain covers the whole child cost.

For a path batch, sum (20) and use Lemma 4.6:

$$\begin{aligned}\sum_{i=1}^{H}\big(\lambda g_i - \ell(m_i)\big) &\ge H\big(\delta(k-2) - 4\lambda - j - 1\big)\\ &\ge H\left(\frac{5}{2} - \frac{7j+65}{k} + \frac{2(j+10)}{k^2}\right) > 0.\end{aligned}$$

The last inequality follows from $7j + 65 \le 2k$ for $j \ge 6$. □

4.4 The comparison bound

Proof of Theorem 4.1. Proposition 4.4 gives consistency and termination of every call. The procedures and their tie rules are deterministic. Suppose the sorter finishes after q comparisons, and write $j = \log k$.

There are at most q/k degree refinements: each deletes k in-node arcs, each comparison inserts at most one arc, and an inserted arc can be deleted only once. Apply Lemma 4.7 to these refinements, the remaining individual refinements, and the complete path batches. This gives

$$\sum_r \ell(m_r) \le \lambda \sum_r g_r + \frac{j+6}{k}\, q. \tag{21}$$

Each comparison decreases $\log Z$ by at most a_η, while a refinement changes it by g_r. Since initially $Z = 1$,

$$\sum_r g_r \le \log Z + a_\eta q.$$

Substitute this into (21) and then into the terminal bound (15):

$$\log(n!) \le n + (\lambda - 1)\log Z + \left(\lambda a_\eta + \frac{j+6}{k}\right) q. \tag{22}$$

The term $(\lambda - 1)\log Z$ is nonpositive. Moreover,

$$\begin{aligned}\lambda a_\eta + \frac{j+6}{k} &\le 1 + \frac{2j + 33/2}{k} + \frac{j+10}{2k^2}\\ &\le 1 + \frac{2j+17}{k},\end{aligned} \tag{23}$$

where the last inequality uses $j + 10 \le k$. Finally,

$$\log(n!) \ge \int_1^n \log x\, dx = n\log n - \frac{n}{\ln 2} + \frac{1}{\ln 2}.$$

Combining this estimate with (22), (23), and $1 + 1/\ln 2 < 3$ proves (3). □

The next section proves the refinement and consistency guarantees used above.

5 Refinement and its guarantees

We now justify the refinement claims used in Section 4. First, any interior anchor preserves consistency with previous answers. Next we give `ChooseAnchor` and prove Lemma 4.2 using the probability estimates from Lemma 3.1. We then establish the in-node DAG bounds, legality of the full procedure, and the distinct-anchor property used to count path refinements.

5.1 Preserving the represented orders

We use the interpretation of the state given in Section 3. Assignments with equal free coordinates, or with a free coordinate equal to an anchor, lie on a finite union of sets of volume zero and are excluded when reading an order. We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

The invariant must continue to hold when refinement removes an arc from an in-node DAG.

Lemma 5.1 (Effective intervals and sections)**.** *Replacing interval bounds by their effective endpoints leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. When these widths are positive, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

Proof. If $y \neq x$ precedes x on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds given by (1) already hold.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \leq \ell_y$ and $r_x \leq r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Increasing θ_x to θ_y strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints ℓ_y, r_y cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds c at its in-neighbors and lower bounds c at its out-neighbors. Upper bounds propagate only backward along arcs, and lower bounds only forward. Any lower bound that previously reached a successor through x is superseded by $c > \ell_x$; any upper bound that reached a predecessor through x is superseded by $c < r_x$. Thus the new effective intervals are

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the in-node DAG is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining in-node DAG, shows that the section has positive volume. □

Lemma 5.2 (Recognizing determined comparisons)**.** *Suppose every node region has positive volume. At the node reached by routing a query, the anchor, effective-interval, and directed-path tests in Algorithm 1 detect every comparison that holds in all represented orders.*

Proof. Distinct child anchors determine their subtrees' order. A free coordinate X_x ranges over its entire effective interval by Lemma 5.1. It can therefore lie on either side of an anchor c exactly when $\ell_x < c < r_x$.

Consider two free items x, y. A directed path fixes their order. If they are incomparable in the in-node DAG, we claim that

$$X_x < X_y \text{ is feasible} \quad \Longleftrightarrow \quad \ell_x < r_y. \tag{24}$$

Necessity follows from $\ell_x < X_x < X_y < r_y$. For sufficiency, add the arc $x \to y$. This preserves acyclicity. It caps the effective upper endpoints of x and its predecessors by r_y, and raises the effective lower endpoints of y and its successors to ℓ_x. These sets are disjoint, since a common vertex would give a path from y to x. Every affected predecessor has lower endpoint at most $\ell_x < r_y$, and every affected successor has upper endpoint at least $r_y > \ell_x$. All effective widths remain positive, so Lemma 5.1 gives a feasible assignment with $X_x < X_y$.

Applying (24) in both directions shows that both answers are possible precisely when the effective intervals overlap. Each feasible local choice extends to a represented total order by choosing feasible assignments in the other nodes and avoiding ties. Hence an undecided local test always leaves both orders possible. □

Lemma 5.3 (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining x at an interior anchor c preserves both properties, including when the child at c already exists. The transcript implies no relation between x and any label already in that child subtree.*

Proof. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the transcript.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

5.2 The volume gained by refinement

Recall that $Z = \prod_v V_v$ and that $f_x(c) = S_x(c)/V_v$ is the density of X_x under the uniform distribution on its node's feasible region. Fixing $X_x = c$ removes one coordinate from the parent; placing x in a child adds an independent unit coordinate there.

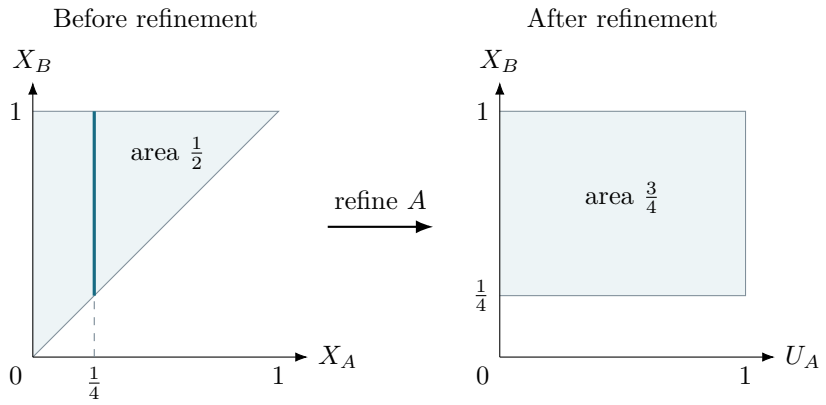


Figure 7: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

Proof of Lemma 4.3. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0, 1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0, 1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0, 1)$ turns this into a rectangle of area 3/4, so refinement multiplies Z by 3/2; see Figure 7. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

5.3 Choosing an anchor

Consider a positive-volume node whose in-node DAG has degree at most k and directed paths of at most $2H$ vertices. Algorithm 3 gives `ChooseAnchor`. It first chooses a closed interval of central positions for x, then selects the shallowest anchor in that interval. For a narrow coordinate, we use the central half of its effective interval. For a wide coordinate, `Marginal` from Lemma 3.1 supplies a finite list of positions approximating the distribution of X_x; we use the interval between the list's first and third quartiles. We will show that either choice gives the density bound required by Lemma 4.2.

Algorithm 3: `ChooseAnchor`(x): choose a position for refinement.

Input: A free item x stored at node v, its effective interval (ℓ_x, r_x), and the fixed parameters R, η.
Output: A dyadic anchor $c \in (\ell_x, r_x)$.

```
W_x ← r_x − ℓ_x
if W_x < 1/R then
    a ← ℓ_x + W_x/4; b ← r_x − W_x/4
else
    (z_1, ..., z_N) ← Marginal(v, x, η), sorted increasingly
    a ← z_⌈N/4⌉; b ← z_⌈3N/4⌉
u ← the root of the implicit anchor tree
while true do
    if c(u) < a then
        u ← the right child of u
    else if c(u) > b then
        u ← the left child of u
    else
        return c(u)
```

An anchor at depth m has $m+1$ fractional binary digits in reduced form. Thus the walk finds the shortest binary fraction in $[a, b]$.

Lemma 5.4 (Shortest anchor in an interval)**.** *For a nondegenerate closed interval $[a, b] \subset (0, 1)$, the walk in Algorithm 3 returns its unique minimum-depth anchor. If its node is u and its depth is m, then $[a, b] \subseteq I(u)$ and $|I(u)| = 2^{-m}$.*

Proof. Initially $[a, b] \subseteq I(u) = [0, 1]$. Whenever $c(u) < a$, the whole interval lies in the right child's interval; whenever $c(u) > b$, it lies in the left child's interval. The rejected anchor and the other subtree contain no candidate in $[a, b]$. The first accepted anchor therefore has minimum depth, and every other anchor of that depth lies outside $I(u)$. The intervals halve at every step and continue to contain $[a, b]$, so the walk must accept before their length becomes smaller than $b - a$. □

5.3.1 Density bounds for central intervals

Suppose x has effective interval (ℓ, r) of width W and at most B other comparable labels. Then

$$f_x(t) \le \frac{B+1}{W}, \qquad f_x(t) \ge \frac{4^{-B}}{W} \quad \left(\ell + \frac{W}{4} \le t \le r - \frac{W}{4}\right). \tag{25}$$

The lower bound applies to every point of the central half. The upper bound limits the probability in a short interval; it will bound the depth of the anchor chosen for a wide item.

Write $F_x(t) = \mathbb{P}(X_x < t)$. We also need a density bound that uses the probability contained in an interval: for any $a \le c \le b$,

$$(b-a) f_x(c) \ge \min\{F_x(c), 1 - F_x(c)\}(F_x(b) - F_x(a)). \tag{26}$$

If both tails at c and the mass of $[a, b]$ are bounded below, this forces the density at c to be a constant times $1/(b-a)$. Appendix A proves (25) by contracting convex sections, and derives (26) from log-concavity.

5.3.2 Choosing an anchor between approximate quartiles

The list returned by `Marginal` may contain repeated values. Write $\widehat{F}_x(t)$ for the fraction strictly below t, and $\widehat{F}_x(t+)$ for the fraction at most t. The guarantee of Lemma 3.1 gives

$$|\widehat{F}_x(t) - F_x(t)| \le \eta, \qquad |\widehat{F}_x(t+) - F_x(t)| \le \eta.$$

The second inequality follows from the first by taking a right limit, since F_x is continuous. Both hold at every real t for this one list.

Lemma 5.5 (Wide anchors)**.** *For a wide item, Algorithm 3 returns an interior anchor c of depth m satisfying*

$$f_x(c) \ge 2^{m-4}, \qquad m < \log \frac{4(B+1)}{W_x}.$$

Proof. For $p \in \{1/4, 3/4\}$, let $Q_p = z_{\lceil pN \rceil}$. Its rank gives

$$\widehat{F}_x(Q_p) \le p \le \widehat{F}_x(Q_p+),$$

including when values are repeated. Consequently

$$p - \eta \le F_x(Q_p) \le p + \eta.$$

Since $\eta < 1/32$, the quartiles are distinct and lie strictly inside (ℓ_x, r_x). Lemma 5.4 supplies an anchor $c \in [Q_{1/4}, Q_{3/4}]$ whose node interval $I(u) = [a, b]$ contains both quartiles. Thus

$$\min\{F_x(c), 1 - F_x(c)\} \ge \frac{1}{4} - \eta > \frac{7}{32}, \qquad F_x(b) - F_x(a) \ge \frac{1}{2} - 2\eta > \frac{7}{16}.$$

As $|I(u)| = 2^{-m}$, (26) gives

$$2^{-m} f_x(c) \ge \frac{7}{32}\frac{7}{16} = \frac{49}{512} > \frac{1}{16},$$

which proves the density guarantee.

The upper density bound in (25) also gives

$$\frac{1}{4} < F_x(b) - F_x(a) \le \frac{B+1}{W_x} 2^{-m}.$$

Rearranging proves the depth bound. □

5.3.3 The anchor guarantee

For a wide item, the preceding lemma gives $\log f_x(c) \ge m - 4$. For a narrow item, the central density bound loses $2B$ in its logarithm. We choose

$$R = 2^{2kB}. \tag{27}$$

Then $W_x < 1/R$ makes $\log(1/W_x)$ large enough to absorb this loss.

Proof of Lemma 4.2. The in-node DAG bounds imply that x has at most B other comparable coordinates, by (8). For a wide item, Lemma 5.5 gives the stronger lower bound $m - 4$. For a narrow item, write $L = \log(1/W_x)$. The selected anchor is in the central half of its effective interval, so (25) gives

$$\log f_x(c) \ge L - 2B.$$

By Lemma 5.4, its node interval contains that central half. Hence $W_x/2 \le 2^{-m}$ and $m \le L+1$. Since $L > 2kB$,

$$\log f_x(c) \ge L - 2B > (1-1/k)L \ge (1-1/k)(m-1) > (1-1/k)m - 4.$$

This proves (9). The same density bound also gives $\log f_x(c) > 2B(k-1) > k$, proving (10). Both cases use a finite deterministic calculation to obtain the central interval, followed by the finite walk of Lemma 5.4. □

The rule also controls the number of bits in a narrow anchor. If the effective endpoints have at most Λ fractional binary digits, their midpoint has at most $\Lambda+1$ and lies in the central half. The chosen anchor has the fewest digits in that interval, so it uses at most $\Lambda+1$ as well.

5.4 Bounds during a comparison call

The preparation loop in Algorithm 1 simplifies the current node before an answer is selected. We check that the in-node DAG bounds hold throughout the call, and that the loop makes only finitely many moves.

Lemma 5.6 (In-node DAG bounds and termination)**.** *Suppose every in-node DAG has degree at most k and directed paths of fewer than $2H$ vertices at the start of a call. These bounds hold throughout the call. Each label is refined at most once, so the call makes at most n refinements.*

Proof. A refinement deletes a vertex and its incident arcs from its parent and adds an isolated vertex to a child. It cannot increase any degree or extend any directed path. Immediately before a new answer is selected, the current in-node DAG has no path on H vertices and the queried free vertices have degree below k. Adding their comparison arc leaves degree at most k. Any new directed path consists of two old paths joined by this arc, so it has fewer than $2H$ vertices. The certainty tests have already excluded a directed path in either direction between the queried items, so the new arc cannot create a cycle. An answer involving an anchor adds only an interval bound.

Once a label is refined, it has an isolated coordinate with interval $(0,1)$ in its child. Until the final answer, every subsequent state change is another refinement. These moves affect interval bounds only at neighbors of the vertex being removed, so they cannot narrow this isolated coordinate or give it an incident arc. It cannot lie on a selected path, have degree k, or be narrow. Thus it cannot be selected again, even if the query is rerouted to its new node. Every restart refines at least one label, giving the bound of n refinements. □

Proof of Proposition 4.4. Initially every node region has positive volume, every order is represented, and the in-node DAG bounds hold. We induct over the changes made by a comparison call. The bounds follow from Lemma 5.6. At a refinement, Lemma 4.2 supplies an interior anchor, and Lemma 5.3 preserves positive volume and the order invariant. This applies to every item of a selected path, including those whose incident arcs have already been removed by earlier moves.

If the local tests determine the answer, it holds in every represented order. Otherwise, the preparation loop ends with wide queried free coordinates. Lemma 3.1 applies to the current positive region; the selected answer retains at least $1/2 - \eta > 0$ of its volume. Recording its arc or interval bound makes that answer true in every represented order and preserves every earlier constraint. This completes the induction.

The probability estimator and anchor search terminate by Lemmas 3.1 and 4.2. Together with Lemma 5.6, this proves termination of the full call. These analytic lemmas apply to arbitrary positive-volume regions with the stated in-node DAG bounds; their proofs are independent of the adversary's history. □

5.5 Why a long path requires deep anchors

Proof of Lemma 4.6. Every in-node arc on the selected path records an answered comparison. The transcript therefore implies an order between every pair of labels on the path. Those implications remain true after the in-node arcs are deleted. By Lemma 5.3, two of these labels cannot be admitted to the same child of their parent in the adversary's tree. The procedure completes all $H = 2^k$ refinements in this parent, so its anchors occupy H distinct nodes of the implicit anchor tree.

The smallest possible sum of depths uses the $2^k - 1$ nodes at depths $0, \dots, k-1$ and one node at depth k. Its value is

$$\sum_{m=0}^{k-1} m2^m + k = (k-2)2^k + k + 2 \geq H(k-2).$$

Every other choice of H distinct nodes has at least this sum of depths. □

6 Computing comparison probabilities

We now implement `Estimate` and the finite marginal calculation used by `ChooseAnchor`. Both average a finite collection of explicit coordinate updates. Their accuracy follows by comparing these updates with an assignment uniformly distributed on the full feasible region.

6.1 A coordinate update with finitely many choices

Let G be the in-node DAG. Compute its effective endpoints ℓ_z, r_z using the full graph, in temporary storage. For a current assignment Y, the interval permitted at coordinate z has endpoints

$$L_z = \max(\{\ell_z\} \cup \{Y_u : u \to z\}), \qquad U_z = \min(\{r_z\} \cup \{Y_u : z \to u\}).$$

Fix a power of two S. For a digit $j \in \{0, \dots, S-1\}$, update

$$Y_z \leftarrow \frac{Y_z + (S-1-j)L_z + jU_z}{S}. \tag{28}$$

The coefficients are nonnegative and sum to one. Thus an assignment satisfying the interval bounds and arc inequalities weakly continues to do so after an update, including when $L_z = U_z$.

This update has a useful exact property. If an assignment is uniform on the feasible region, then, conditional on its other coordinates, Y_z is uniform on (L_z, U_z). Its normalized position $r = (Y_z - L_z)/(U_z - L_z)$ is uniform on $(0, 1)$, and (28) sends it to

$$r' = \frac{r + j}{S}.$$

For a fixed digit, this is uniform on the corresponding one of the S equal subintervals. Choosing the digit uniformly therefore makes r' uniform on $(0, 1)$ again. Each update preserves the uniform distribution on the whole feasible region.

6.2 The finite calculation

Fix a topological order of G. For a single queried coordinate x, retain its radius-ρ neighborhood in the undirected graph. For a pair query, retain the union of both neighborhoods. Keep every arc between retained vertices and copy their effective endpoints from the full graph. These endpoints carry interval restrictions transmitted by paths outside the retained neighborhood.

Initialize the retained coordinates at their effective lower endpoints. These values satisfy all retained arc inequalities weakly. One pass through the retained vertices in topological order is a *sweep*. Perform t sweeps, using (28) at each visit. The algorithm enumerates every assignment of digits to these updates. If b' vertices are retained, there are

$$N = S^{b't} \tag{29}$$

such computations. Algorithm 4 records their final queried values, including repetitions.

Algorithm 4: `FiniteDigits`: compute the finite list of queried values.

Input: The retained in-node DAG, its copied effective bounds, queried coordinates T, and integers S, t.
Output: A list $\mathcal{L}$ of final values at T, one entry per digit assignment.
$\mathcal{L} \leftarrow$ the empty list
foreach *assignment of digits* $j_{z,s} \in \{0, \dots, S-1\}$ *to the* $b't$ *updates* **do**
  $Y_z \leftarrow \ell_z$ for every retained vertex z
  for $s = 1, \dots, t$ **do**
    foreach *retained vertex* z *in topological order* **do**
      Compute L_z, U_z from its current neighbors and copied bounds
      $Y_z \leftarrow (Y_z + (S - 1 - j_{z,s})L_z + j_{z,s}U_z)/S$
  Append $(Y_z)_{z \in T}$ to $\mathcal{L}$
return $\mathcal{L}$

For a probability query, `Estimate` returns the fraction of entries satisfying the requested strict comparison. A pair already ordered by a directed path is answered exactly before this calculation. For a wide coordinate x, `Marginal` returns the single-coordinate list $\mathcal{L}_x$. Define its strict empirical distribution function by

$$\widehat{F}_x(c) = \frac{\#\{y \in \mathcal{L}_x : y < c\}}{N}. \tag{30}$$

Its right limit $\widehat{F}_x(c+)$ counts values at most c. The list may contain ties, so these two quantities can differ.

Theorem 6.1 (Finite probability estimates)**.** *Let $d, h \geq 1$ be integers. Suppose G has degree at most d, its directed paths have at most h vertices, and its feasible region $K \subseteq (0,1)^{V(G)}$ has positive volume. For rational parameters $R \geq 1$ and $0 < \varepsilon < 1/4$, computable choices of S, t, ρ, depending only on these parameters, have the following properties.*

1. *If $W_x \geq 1/R$, the single list $\mathcal{L}_x$ satisfies, for every real c,*
$$|\widehat{F}_x(c) - F_x(c)| \leq \varepsilon, \qquad |\widehat{F}_x(c+) - F_x(c)| \leq \varepsilon, \tag{31}$$
where $F_x(c) = \mathbb{P}(X_x < c)$ for X uniform on K.
2. *For an incomparable pair x, y with at least one of W_x, W_y at least $1/R$, the fraction of entries satisfying $Y_x < Y_y$ differs from $\mathbb{P}(X_x < X_y)$ by at most ε.*

The calculation retains at most

$$b = 2(d+1)^{\rho+1} \tag{32}$$

vertices and enumerates at most S^{bt} digit assignments.

6.3 Why the finite calculation is accurate

The full uniform distribution is preserved by a random digit update. We compare it with the local calculation started at the lower endpoints, using the same digits wherever a vertex is retained. The following argument bounds the probability that their queried coordinates differ by more than a prescribed amount.

Proof of Theorem 6.1. Comparing two computations. Start X uniformly on K and Y at the effective lower endpoints of the retained graph. Update the full graph in topological order, and use the restriction of that order for Y. Share the digit at each retained vertex. Digits at different vertices and sweeps are independent and uniform in $\{0, \dots, S-1\}$. The full assignment X remains uniform on K. Every digit assignment for Y has probability $1/N$, so its final queried values have exactly the empirical distribution of the computed list.

The decrease after one sweep. Write $\Delta_z = |X_z - Y_z|$. Superscripts old and new refer to the start of a sweep and the value after a coordinate's update. If all neighbors of z are retained, the shared digit j_z gives

$$\Delta_z^{\text{new}} \le \frac{1}{S}\Delta_z^{\text{old}} + \frac{S-1-j_z}{S} \max_{u \to z} \Delta_u^{\text{new}} + \frac{j_z}{S} \max_{z \to u} \Delta_u^{\text{old}}, \tag{33}$$

with an empty maximum equal to zero.

Let $B_h(z)$ be the vertices at undirected distance at most h from z, and suppose they are all retained. Put $D = \max_{u \in B_h(z)} \Delta_u^{\text{old}}$ and $a_u = (S-1-j_u)/S$. At z, (33) gives

$$\Delta_z^{\text{new}} \le (1-a_z)D + a_z \max_{u \to z} \Delta_u^{\text{new}}.$$

Follow a predecessor attaining the maximum, and continue backwards to a source. This path has at most h vertices; all its vertices and their successors lie in $B_h(z)$. At the source the predecessor term is zero. Substitution along the path bounds the new discrepancy by $\left(1 - \prod_u a_u\right)D$.

Write $\Delta_z(s)$ for the discrepancy after s sweeps. Call the pair (z, s) *bad* if, during sweep s, the digit at z or any ancestor of z equals $S-1$. Otherwise all factors a_u in the preceding expansion are at least $1/S$, and there are at most h of them. Put $\beta = 1 - S^{-h}$. Consequently,

$$\Delta_z(s) \le \kappa_z(s) \max_{u \in B_h(z)} \Delta_u(s-1), \qquad \kappa_z(s) = \begin{cases} 1, & (z,s) \text{ is bad,} \\ \beta, & \text{otherwise.} \end{cases} \tag{34}$$

Following the possible dependencies. Both $B_h(z)$ and the set consisting of z and its ancestors have at most M vertices. Choose

$$M = (d+1)^{h+1}, \qquad S = \text{the least power of two at least } 16M^3. \tag{35}$$

The probability that (z, s) is bad is at most $M/S \le 1/(16M^2)$.

For even t, iterating (34) traces sequences $z_t = z, z_{t-1}, \dots, z_0$ with $z_{s-1} \in B_h(z_s)$. There are at most M^t sequences. If $B_{ht}(z)$ is retained, each $B_h(z_s)$ used at sweep $s \ge 1$ is retained: its vertices are at distance at most $(t-s+1)h \le th$ from z. Thus the expansion is valid throughout, including at the boundary, where only initial discrepancies occur. Since those initial discrepancies are at most one,

$$\Delta_z(t) \le \max_{z_t = z,\, z_{s-1} \in B_h(z_s)} \prod_{s=1}^{t} \kappa_{z_s}(s). \tag{36}$$

For a fixed sequence, the events that its pairs are bad use independent digits from different sweeps. The probability that at least $t/2$ pairs are bad is at most

$$2^t \left(\frac{1}{16M^2}\right)^{t/2} = \left(\frac{1}{2M}\right)^t.$$

A union bound over the at most M^t sequences gives probability at most 2^{-t} that any sequence has this many bad pairs. Otherwise every product in (36) has at least $t/2$ factors at most β. Hence

$$\mathbb{P}(\Delta_z(t) > \beta^{t/2}) \le 2^{-t}. \tag{37}$$

From coordinate errors to comparison errors. Put $C = M + 1$ and choose

$$\delta = \frac{\varepsilon}{8CR}, \qquad t = 2\left(S^h \lceil \log(1/\delta) \rceil + \lceil \log(4/\varepsilon) \rceil\right), \qquad \rho = ht. \tag{38}$$

The inequality $(1 - v)^{-r} \geq 1 + rv$ gives $\beta^{S^h} \leq 1/2$. Therefore $\beta^{t/2} \leq \delta$ and $2^{-t} \leq \varepsilon/4$. Each queried coordinate satisfies

$$\mathbb{P}(\Delta_z(t) > \delta) \leq \varepsilon/4.$$

For an incomparable pair, choose x with $W_x \geq 1/R$. There are at most M other coordinates comparable with x, so Corollary A.2 gives

$$\mathbb{P}(|X_x - X_y| \leq 2\delta) \leq 4CR\delta = \varepsilon/2.$$

Outside this event and the two coordinate-error events, the strict comparisons in X and Y agree, even if Y can have ties. Their probabilities therefore differ by at most $\varepsilon/2 + 2(\varepsilon/4) = \varepsilon$.

For a threshold c, disagreement requires either $\Delta_x > \delta$ or $|X_x - c| \leq \delta$. Its probability is at most $\varepsilon/4 + 2CR\delta = \varepsilon/2$. This bound applies to every real c, using the same computed list. It proves the strict-CDF bound in (31). Taking right limits and using the continuity of F_x proves the other bound, including at atoms of the empirical distribution.

Finally, each radius-ρ neighborhood has at most $1 + d + \cdots + d^\rho \leq (d + 1)^{\rho+1}$ vertices. This gives (32). The choices of S, t, ρ are computable from the stated parameters. □

6.4 Exact arithmetic for the digit calculations

Suppose the retained effective endpoints have at most Λ fractional binary digits. Each update in (28) takes maxima and minima, forms an integer linear combination, and divides by S. Because S is a power of two, it adds at most $\log S$ fractional bits. After all $b't$ updates in a computation, the precision is at most

$$\Lambda + b't \log S. \tag{39}$$

All values remain in $[0, 1]$, so the corresponding integer numerators need at most one additional bit. The digits and tallies have at most $\log S$ and $b't \log S + 1$ bits, respectively. The comparison rule compares the exact tally with $N/2$; a marginal calculation retains the exact final values and their multiplicities. These bounds will give polynomial time under the explicit enumeration budget in Section 7.

Finite computations can contain equal coordinates. Accordingly, a pair estimate counts the strict event $Y_x < Y_y$, and the comparison rule uses its complement for the opposite answer. The continuous probabilities of the two orders sum to one, so this complement has the same error bound.

Proof of Lemma 3.1. The strict inequalities defining a feasible region make it open, so a feasible assignment gives positive volume. Computing the effective endpoints in temporary storage preserves the region by Lemma 5.1. Apply Theorem 6.1 with $d = k$, $h = 2H$, and $\varepsilon = \eta$. It gives the required probability estimates and the uniform empirical-CDF bound for `Marginal`. A pair already ordered by a directed path is answered exactly. All other calculations enumerate finitely many digit assignments using exact dyadic arithmetic. □

The next section chooses the parameters as a function of n and bounds the work in every comparison call.

7 A uniform polynomial-time implementation

For each fixed degree threshold k, the preceding sections give a finite deterministic algorithm. Its local calculations depend on k, while its interval endpoints depend on the comparison history. We first bound the size of the maintained state, then choose k slowly enough that the local calculations also take polynomial time. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

7.1 Bounding the state throughout the interaction

The certainty tests in Algorithm 1 recognize every comparison fixed by the represented orders (Lemma 5.2). Since those orders respect all earlier answers, a previously answered pair is recognized before any refinement begins. Such a call returns without changing the state. Every call that changes the state therefore compares a previously unanswered pair, so there are at most $\binom{n}{2}$ such calls, however often the sorter repeats a comparison.

Lemma 5.6 bounds the number of refinements in one call by n. Consequently their total number P, including those in a call still in progress, satisfies

$$P \le n\binom{n}{2}. \tag{40}$$

Each refinement creates at most one node in the adversary's tree, so that tree has at most $P+1$ nodes. The n item records are distributed among these nodes, and each in-node DAG has degree at most k. The algorithm stores this tree, its records, and its in-node DAGs.

We store an anchor by its path in the implicit anchor tree. A path of length m gives its interval and position by m halvings of $[0,1]$; the position has $m+1$ fractional binary digits. Anchor comparisons use these exact numerical positions.

For a wide item, Lemma 5.5 bounds its anchor precision by a function of k alone. We may use

$$K_0(k) = \lceil \log(16(B+1)R) \rceil + 2.$$

For a narrow item, suppose the stored endpoints have at most Λ fractional digits. The midpoint of its effective interval has at most $\Lambda+1$ digits and lies in the central half. The chosen fraction has the fewest digits there, so it uses at most $\Lambda+1$ digits as well. Effective endpoints are maxima and minima of stored bounds, which add no digits. After P refinements,

$$\Lambda \le K_0(k) + P. \tag{41}$$

7.2 An explicit budget for the finite calculations

Apply the construction in Section 6 with $d = k$, $h = 2H$, and error η. Write S_k for its number of digit choices, t_k for the number of sweeps, and b_k for the bound on retained vertices in (32). The number of digit sequences in any one calculation is at most

$$E_k = S_k^{b_k t_k}. \tag{42}$$

This explicit integer will determine which parameters we can afford.

For powers of two $k = 64, 128, \ldots$, a fixed program writes the complete parameter tuple and E_k in binary. On input size n, simulate this program for n steps. Among the tuples it has completed, choose the largest k satisfying

$$E_k \le n. \tag{43}$$

Use these parameters for the entire interaction. If no tuple qualifies, answer all comparisons according to the fixed label order.

The initialization takes polynomial time: the simulation writes at most n bits, and its completed tuples can be parsed and compared within a polynomial bound. For every fixed k, the program finishes its tuple in some finite time $T(k)$. Once $n \geq \max\{T(k), E_k\}$, that tuple is available and meets the budget. Hence the chosen $k(n)$ tends to infinity.

The budget controls both the number and the size of the computations. There are at most n digit sequences per estimate or marginal calculation. Each sequence has at most $b_k t_k \leq \log n$ updates, and these updates add at most $b_k t_k \log S_k = \log E_k \leq \log n$ fractional bits. Tallies have at most $\log n + 1$ bits. A marginal calculation stores at most n final values; sorting this list gives the two quartiles.

The constants in (38) also give $K_0(k) \leq \log E_k$: here $M = B$, and the chosen number of sweeps exceeds $\lceil \log(8(B+1)R/\eta) \rceil$, which is larger than $K_0(k)$. Thus (40) and (41) give $\Lambda = O(n^3)$. Every update uses exact arithmetic on polynomially many bits. The shortest-fraction walk takes at most the bit length of an available candidate: the midpoint in the narrow case, or either empirical quartile in the wide case. Its comparisons therefore also have polynomial bit cost.

A comparison call makes at most n refinements, each requiring at most one marginal calculation, followed by at most one probability estimate. Routing queries, computing effective endpoints, and testing reachability use polynomial time in the stored state. Directed-path lengths are computed in reverse topological order; if $H > n$, no path can require refinement. All tie rules can be implemented by scanning labels in order. Thus each comparison is answered within one fixed polynomial number of bit operations, independent of the chosen parameters and of the length of the preceding interaction.

Proof of Theorem 1.1. Initialization and every comparison call are deterministic and take polynomial time. For all sufficiently large n, a parameter tuple qualifies. Theorem 4.1 gives legality and forces

$$q \geq \frac{n \log n - 3n}{1 + (2 \log k(n) + 17)/k(n)} = (1 - o(1)) n \log n,$$

since $k(n) \to \infty$. The fixed-order rule for the remaining input sizes is also deterministic and legal. □

A Coordinate densities

We prove the density bounds used to choose anchors and to turn coordinate approximation into probability approximation. The first bounds follow by fixing the incomparable coordinates and comparing sections of the remaining convex region. Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \neq x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \mathrm{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\mathrm{vol}_N K}, \qquad F_x(t) = \int_{-\infty}^{t} f_x(s)\, ds.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.1 (Coordinate density bounds)**.** *Suppose K has positive volume, and x has at most B other comparable labels and effective interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \le \frac{B+1}{W}, \tag{44}$$

$$f_x(t) \ge \frac{4^{-B}}{W} \qquad (\ell + W/4 \le t \le r - W/4). \tag{45}$$

These bounds also hold after fixing any set of labels incomparable with x, for every resulting slice of positive volume.

Proof. We may first fix any prescribed set of coordinates incomparable with x, at values leaving a positive-volume slice; write f_x for its coordinate density. Fixing a coordinate turns its incident inequalities into interval bounds. These bounds propagate along directed paths, so fixing an incomparable coordinate leaves x's effective endpoints unchanged. By Lemma 5.1, its projection interval remains (ℓ, r). The same argument applies successively as more incomparable coordinates are fixed.

Now fix all the remaining coordinates incomparable with x. For each choice leaving a nonempty slice, the remaining feasible region C is open and convex, has dimension $d \le B+1$, and projects onto (ℓ, r) in coordinate x. Write $S(t)$ for its section volume at $x = t$ and $g(t) = S(t)/\operatorname{vol} C$ for the resulting density.

Choose points in $\overline{C}$ whose x-coordinates are ℓ and r. Such points exist because C is bounded. For $\ell < t < s < r$, join every point of the section at t to the point at r. Since C is open and convex, the image at $x = s$ lies in the section of C there. Each of its $d-1$ directions is scaled by $(r-s)/(r-t)$, so

$$S(s) \ge \left(\frac{r-s}{r-t}\right)^{d-1} S(t) \qquad (t < s). \tag{46}$$

Joining instead to the point at ℓ gives

$$S(s) \ge \left(\frac{s-\ell}{t-\ell}\right)^{d-1} S(t) \qquad (s < t). \tag{47}$$

Normalize these inequalities and integrate over s on both sides of t:

$$1 \ge g(t) \left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^{d-1} ds + \int_t^r \left(\frac{r-s}{r-t}\right)^{d-1} ds \right) = \frac{W g(t)}{d}.$$

Thus $g(t) \le d/W \le (B+1)/W$.

For the lower bound, choose t_0 with $g(t_0) \ge 1/W$, which exists because g integrates to one over an interval of length W. For any $s \in [\ell + W/4, r - W/4]$, the appropriate ratio in (46) or (47) is at least $1/4$. Consequently

$$g(s) \ge 4^{-(d-1)} g(t_0) \ge \frac{4^{-B}}{W}.$$

The case $s = t_0$ satisfies the same bound directly.

Finally, average over the additional incomparable coordinates fixed in the second paragraph. If ω denotes their values, C_ω the remaining region, and g_ω its coordinate density, integration of each section gives the pointwise identity

$$f_x(t) = \frac{\int \operatorname{vol}(C_\omega) g_\omega(t)\, d\omega}{\int \operatorname{vol}(C_\omega)\, d\omega}.$$

Empty slices contribute zero. The bounds on g_ω are uniform in ω, so they hold for f_x at every t. This proves both the unconditional bounds and the bounds after the initial conditioning. □

Corollary A.2 (A bound for small differences)**.** *If y is incomparable with x, or Y is a fixed number, then for every fixed interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \le \frac{(B+1)a}{W}, \tag{48}$$

where $Y = X_y$ in the first case.

Proof. In the first case, condition on X_y. Lemma A.1 bounds the conditional density of x by $(B+1)/W$. Integrating over the translate $I + X_y$ and then averaging over X_y proves the claim. For fixed Y, apply the unconditional density bound. □

To bound the density at an anchor selected by the search, we use log-concavity. A nonnegative function g is *log-concave* if

$$g((1-\theta)u + \theta v) \ge g(u)^{1-\theta} g(v)^{\theta} \qquad (0 < \theta < 1).$$

Prékopa's marginal theorem states that integrating out coordinates of a log-concave probability density preserves log-concavity [11, Theorem 8].

Lemma A.3 (Log-concavity of coordinate densities)**.** *Every coordinate density f, extended by zero outside its effective interval, is log-concave. In particular, for $u \le c \le v$,*

$$f(u)f(v) \le f(c)f(u+v-c). \tag{49}$$

Proof. The feasible region K is convex, so its uniform density, equal to $1/\operatorname{vol} K$ on K and zero elsewhere, is log-concave. Prékopa's theorem therefore applies to the coordinate density obtained by integrating out the other coordinates.

For the displayed inequality, the cases $c = u$, $c = v$, and $f(u)f(v) = 0$ are immediate. Otherwise put $\theta = (c-u)/(v-u)$. The points c and $u+v-c$ are the convex combinations of u, v with weights $(1-\theta, \theta)$ and $(\theta, 1-\theta)$, respectively. Multiplying the two log-concavity inequalities proves the claim. □

Lemma A.4 (Density from mass on both sides)**.** *For a coordinate density f, its cumulative distribution F, and $a \le c \le b$,*

$$(b-a)f(c) \ge \min\{F(c), 1-F(c)\}\big(F(b) - F(a)\big). \tag{50}$$

Proof. Lemma A.3 gives $f(u)f(v) \le f(c)f(u+v-c)$ for $u \le c \le v$. For fixed $v \ge c$, integration over $u \le c$ gives

$$F(c)f(v) \le f(c) \int_{-\infty}^{c} f(u+v-c)\, du \le f(c),$$

because the integral of a density over any interval is at most one. Integrating this inequality over $c \le v \le b$ yields

$$F(c)\big(F(b) - F(c)\big) \le (b-c)f(c).$$

Likewise, for fixed $u \le c$, integration over $v \ge c$ gives $f(u)(1-F(c)) \le f(c)$. Integrating over $a \le u \le c$ yields

$$\big(F(c) - F(a)\big)\big(1 - F(c)\big) \le (c-a)f(c).$$

Add these inequalities and replace the two tail factors on the left by their minimum. □

B Parameter reference

Table 1 collects the parameters of the adversary. The last two columns distinguish where a parameter's role is introduced from where its value is chosen and explained. All choices are fixed before the first query. The constants used to bound the estimator's neighborhood are given in (35) and (38); its size is bounded by (32).

Table 1: Parameters and their roles. Section references lead to the explanations in the main text.

Symbol	Value	Role	Introduced	Choice
k	Power of two, $k \geq 64$	Refine a queried free item when it has this many neighbors.	§3.2	§4
H	2^k	Number of vertices refined in a selected directed path.	§3.2	§4.3
B	$(k+1)^{2H+1}$	Bounds the number of other vertices comparable with one vertex in its in-node DAG.	§4.1	§4.1
R	2^{2kB}	Sets the width threshold: a coordinate is wide when $W_x \geq 1/R$.	§3.2	§5.3.3
η	$1/(100k)$	Maximum additive error in a probability estimate.	§3.2	§4.1

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.

[11] András Prékopa. *On logarithmic concave measures and functions.* Acta Scientiarum Mathematicarum 34, 335–343, 1973. Publisher's paper.